

AgriMarket Profit Optimizer

Report on The Code

Problem Statement

Farmers and traders often face challenges identifying the most profitable markets to sell their produce. Transportation costs, varying market prices, and geographical distances make it difficult to determine the best option. This lack of clear insights can lead to reduced profits or losses. There is a need for a tool that dynamically analyses market data, incorporates transportation costs, and recommends the most profitable market for selling agricultural commodities.

Objective of this Code

The primary objective of this code is to assist users in identifying the best market for selling their commodities to maximize profit. The program achieves this by:

1. Calculating the selling price in the user's chosen market.
2. Comparing potential profits across different markets after factoring in transportation costs.
3. Highlighting the most profitable market for selected commodity and variety.

All this is made possible by analysing India Commodity Wise Mandi Dataset which has been retrieved from Kaggle.

About the Dataset

This is a time series dataset, for 325 different commodities sold across all the mandis/APMC markets across India. Data is available between the date range of Jan 1st, 2000 to Feb 1st, 2024.

Each file has the following columns:

- State Name
- District Name
- Market Name
- Variety
- Group
- Arrivals (Tonnes)
- Min Price (Rs./Quintal)
- Max Price (Rs./Quintal)
- Modal Price (Rs./Quintal)
- Reported Date

This Dataset can be accessed by clicking [here](#).

Features

1. Data Loading:

- The `load_data` function loads all `.csv` files from the specified dataset folder, skips empty files, and concatenates them into a single Pandas DataFrame.
- It ensures that only non-empty datasets are processed, avoiding errors.

2. Filtering Latest Data:

- The `get_latest_data` function selects records corresponding to the most recent date in the dataset, ensuring the use of up-to-date information.

3. Interactive User Input:

- Users are guided to select:
 - i. State
 - ii. District
 - iii. Market
 - iv. Commodity
 - v. Variety
- This step-by-step selection process ensures clarity and prevents errors.

4. Distance Calculation:

- The `calculate_distance` function determines the geographical distance (in kilometers) between two cities using the `geopy` library.
- It handles cases where coordinates might be missing or when the user's market is already the best choice.

5. Profit Calculation:

- The script calculates:
 - i. Selling price for the user's chosen market.
 - ii. Transportation costs based on distance and quantity.
 - iii. Net selling price for all markets relative to the selected one.
- This is used to determine the most profitable market.

6. Recommendation:

- The script identifies the market offering the highest profit, accounting for transportation costs.
- It provides detailed feedback, including the name of the best market, profit, and distance.

Libraries and Modules used

The code leverages several Python libraries and modules to handle data, perform calculations, and interact with the user. Below is an explanation of each library or module and its role in the program:

1. OS

- Purpose:
 - Provides functions to interact with the operating system.
- Usage in the Code:
 - Accesses files in the specified dataset folder.
 - Filters files based on their extension (`.csv`) and checks for empty files using `os.stat`.
- Key Methods Used:
 - `os.listdir`: Lists all files in the folder.
 - `os.path.join`: Combines folder path and file name.
 - `os.stat`: Checks file size to skip empty files.

2. pandas (pd)

- Purpose:
 - Used for efficient data manipulation and analysis.
- Usage in the Code:
 - Reads and combines CSV files into a single DataFrame.
 - Filters data dynamically based on user selections (e.g., state, district, market).
 - Computes statistical information like maximum dates and unique values.
 - Performs calculations such as profit margins and updates data columns.
- Key Functions Used:
 - `pd.read_csv`: Reads data from CSV files.
 - `pd.concat`: Concatenates multiple DataFrames into one.
 - `DataFrame.loc`: Filters rows or updates columns.

3. geopy

- Modules Imported:
 - `geopy.geocoders.Nominatim`
 - `geopy.distance.geodesic`
- Purpose:
 - Provides tools for geocoding and calculating distances between geographical locations.
- Usage in the Code:
 - Nominatim is used to obtain geographical coordinates (latitude and longitude) of districts or markets.
 - geodesic calculates the distance between two geographical points.
- Key Features:
 - Helps determine the transportation cost based on distances between districts.

The combination of these libraries enables the program to manage large datasets, perform complex calculations, and offer actionable insights to the user with minimal effort.

Key Functions Used in the Code

1. load_data(folder_path)

This function is responsible for loading all the CSV files from the specified folder path (folder_path). It scans the folder, identifies files with a .csv extension, and attempts to read them into a Pandas DataFrame. Empty files are skipped to prevent errors. If a file contains valid data, it is appended to a list of DataFrames. Finally, all the DataFrames are concatenated into a single DataFrame. This ensures that all market data is consolidated for further analysis. The function also ensures the script does not crash by handling empty datasets gracefully.

```
def load_data(folder_path):
    files = [os.path.join(folder_path, f) for f in os.listdir(folder_path) if f.endswith(".csv")]
    dataframes = []

    for file in files:
        if os.stat(file).st_size == 0:
            continue
        try:
            df = pd.read_csv(file)
            if not df.empty:
                dataframes.append(df)
        except pd.errors.EmptyDataError:
            continue
    print("DATA HAS BEEN LOADED SUCCESSFULLY")
    return pd.concat(dataframes, ignore_index=True)
```

2. get_latest_data(df)

This function filters the loaded dataset to include only the most recent data based on the Reported Date column. It first identifies the latest date present in the dataset and then extracts rows corresponding to this date. This step ensures that the analysis is conducted using up-to-date information, making the recommendations relevant for the current market conditions.

```
def get_latest_data(df):
    latest_date = df["Reported Date"].max()
    return df[df["Reported Date"] == latest_date]
```

3. select_option(options, prompt)

This function simplifies the process of user interaction by providing a menu-based interface. It takes a list of options and a prompt as input and displays the options to the user. The user selects an option by entering the corresponding number. The function validates the input and returns the selected option. This structured approach prevents errors and makes the script user-friendly.

```
def select_option(options, prompt):
    print(prompt)
    for i, option in enumerate(options, 1):
        print(f"{i}. {option}")
    choice = int(input("Enter the number of your choice: "))
    return options[choice - 1]
```

4. calculate_distance(city1, city2)

This function calculates the geographical distance between two cities using the Geopy library. It defines a helper function, `get_coordinates(city)`, which retrieves the latitude and longitude of a given city using Geopy's `geocode` function. Once the coordinates of both cities are obtained, the geodesic distance between them is calculated. If the coordinates are missing or cannot be retrieved, the function prints a message and returns infinity (`inf`) to indicate that the distance cannot be calculated. This function is crucial for factoring in transportation costs between markets.

```
def calculate_distance(city1, city2):
    geolocator = Nominatim(user_agent="city_distance_calculator")

    def get_coordinates(city):
        location = geolocator.geocode(city)
        if location:
            return (location.latitude, location.longitude)
        return None

    coords_1 = get_coordinates(city1)
    coords_2 = get_coordinates(city2)

    if coords_1 and coords_2:
        return geodesic(coords_1, coords_2).kilometers
    else:
        print("Could not calculate distance due to missing coordinates or if the best market is your own district.")
        return float("inf")
```

5. calculate_transport_cost(distance_km, weight_kg)

This function calculates the transportation cost based on the distance between the user's district and the target market's district, as well as the weight of the commodity being transported. The formula used is:

$\text{Cost} = \text{Distance (in km)} \times \text{Weight (in kg)} \times \text{Cost per kg per km}$

The `TRANSPORTATION_COST_PER_KG_PER_KM` constant, set at Rs. 3.6, is used in the calculation. This function is vital for assessing the net profit after deducting transportation expenses.

```
def calculate_transport_cost(distance_km, weight_kg):
    return distance_km * weight_kg * TRANSPORTATION_COST_PER_KG_PER_KM
```

6. main()

The main function is the core of the script, orchestrating the entire workflow. It begins by allowing the user to select their state, district, market, commodity, and variety through sequential calls to

select_option. The user also inputs the quantity of the commodity they wish to sell. The function then calculates the selling price at the user's selected market using the modal price and quantity.

Next, the function iterates through all markets within the same state where the selected commodity and variety are available. For each market, it calculates the net selling price. If the market is not the user's selected market, it factors in transportation costs using calculate_distance and calculate_transport_cost. The profit difference relative to the user's chosen market is computed for each market, and the results are stored in a new column, Profit Relative to Chosen Market.

Finally, the function identifies the market with the highest profit relative to the user's market. If the user's market is the most profitable, it congratulates them. Otherwise, it provides details about the best market, including its name, profit difference, and distance from the user's district.

```
def main():

    states = data["State Name"].unique().tolist()
    state = select_option(states, "Select your state:")

    districts = data[data["State Name"] == state]["District Name"].unique().tolist()
    district = select_option(districts, "Select your district:")

    markets = data[data["District Name"] == district]["Market Name"].unique().tolist()
    market = select_option(markets, "Select your market:")

    commodities = data[data["Market Name"] == market]["Group"].unique().tolist()
    if not commodities:
        print("No commodities available in the selected market.")
        return
    commodity = select_option(commodities, "Select the commodity you want to sell:")

    market_data = data[(data["Market Name"] == market) & (data["Group"] == commodity)]
    if market_data.empty:
        print("No data available for the selected market and commodity.")
        return

    varieties = market_data["Variety"].unique().tolist()
    variety = select_option(varieties, "Select the variety of the commodity:")

    quantity = float(input("Enter the quantity (in kg) you want to sell: "))

    main_data = data[(data["State Name"] == state) &
                     (data["Group"] == commodity) &
                     (data["Variety"] == variety)].copy()

    print("Calculating best market options...")
    profits = []

    selling_price_user_market = (market_data.loc[market_data["Market Name"] == market, "Modal Price (Rs./Quintal)"].values[0] * quantity / 100)
    print(f"Selling price at your chosen market: Rs. {selling_price_user_market:.2f}")

    profits = []

    for _, row in main_data.iterrows():
        if row["Market Name"] == market:
            net_selling_price = row["Modal Price (Rs./Quintal)"] * quantity / 100
        else:
            distance = calculate_distance(district, row["District Name"])
            transport_cost = calculate_transport_cost(distance, quantity)
            net_selling_price = (row["Modal Price (Rs./Quintal)"] * quantity / 100) - transport_cost

        profit_difference = net_selling_price - selling_price_user_market
        profits.append(profit_difference)

    main_data.loc[:, "Profit Relative to Chosen Market"] = profits
    best_market = main_data.loc[main_data["Profit Relative to Chosen Market"].idxmax()]

    if district == best_market["District Name"]:
        print(f"The best market to sell your produce is your own market in {district}, Congratulations!")
    else:
        print(f"Best market for maximum profit relative to your chosen market: {best_market['Market Name']}")
        print(f"Profit relative to your market: Rs. {best_market['Profit Relative to Chosen Market']:.2f}")

        distance = calculate_distance(district, best_market["District Name"])
        if distance != float("inf"):
            print(f"Distance to the market: {distance:.2f} km.")
```

Sample Outputs and Access to Code

- Sample outputs of this code can be accessed [here](#)
- To access the entire code, please click [here](#)

Limitations of this code

1. **Dependence on Geolocation Accuracy:**
 - If city coordinates are unavailable or incorrect, the script cannot calculate distances, leading to potential errors in output.
2. **Dataset being inconsistent:**
 - For implementing this code, the best open-source dataset has been used but still, the dataset has some inconsistencies. Some of these are unavailable modal prices for some commodities, and a lot of empty files and data for some major commodities missing at a specific market which leads to inaccuracies giving user their own market as output most of the time.
3. **The Dataset is large and slow to process:**
 - The dataset is very large with its rows reaching over 50 million, which makes its handling difficult and a slow process.

Submitted by:

1. **Sarthak Mehta**

- Roll No. : 24103143
- Group: B2b

2. **Sandeep Yadav**

- Roll No. 24103141
- Group: B2b